

Visualisierung kinematischer Kennwerte

Philipp Mikos

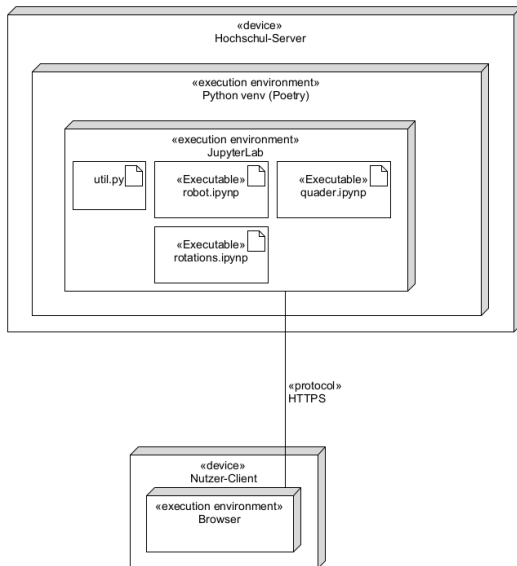
May 7, 2025

- 1 Einleitung
- 2 Anforderungen
- 3 Aufbau
- 4 Entwicklungsprozess
- 5 Evaluation
- 6 Ausblick auf die Bachelorarbeit

- Entwicklung von Framework zur Visualisierung kinematischer Kennwerte für die Lehre
- abstrakte mathematische Konzepte für Studenten schwierig zu durchdringen
- Besonderer Fokus: Rotation von Körpern
- Werkzeug für Lehrpersonen zum erstellen von Übungsszenarien

- Python-Schnittstelle
- Nutzung innerhalb von JupyterLab
- nach möglichkeit Visualisierung innerhalb von JupyterLab
- Transformationen von Körpern im Raum Visualisieren (insbesondere Rotation)
- Darstellungsmöglichkeiten (Euler, Matrix, Quaternionen)

- keine lokale Installation
- komfortable erstellung und Anwendung von Übungsaufgaben
- Kompatibel mit Sympy
- einfache Bewegungsabläufe Radgetriebender Roboter



Weitere Abhängigkeiten:

- Numpy
- SymPy
- SciPy
- pythreejs(3D Rendering)
- ipywidgets(GUI-Elemente)

util.py:

- Zentrale Datei util.py mit Kern- und Hilfsfunktionen
- Unterstützung geometrischer Transformationen und Koordinatensysteme
- Klasse Environment für konfigurierbare 3D-Szene mit Kamera, Licht und Gitter
- Funktion move() zur Simulation von Roboterbewegungen in der Ebene

Beispiel-Notebooks: `rotations.ipynb`

- Würfel der interaktiv per GUI-Elementen transformiert werden kann
- Möglichkeiten des Frameworks veranschaulichen
- Grundtransformationen nachzuvollziehbar machen

Beispiel-Notebooks: **quader.ipynb**

- Quader der anhand von Rotationsmatrizen rotiert werden soll
- Orientiert sich an einer Klausuraufgabe im Modul Robotik
- Visuelle Simulation der Rotationen

Beispiel-Notebooks: `robot.ipynb`

- Veranschaulichung einer Steuerungsaufgabe
- Orientiert sich an einer Übungsaufgabe im Modul Robotik
- Nutzer kann beobachten ob der Roboter dem gewünschten Pfad folgt

Wahl der Rendering API: OpenGL

- Plattformunabhängig (Windows, macOS, Linux)
- Bewährt in 3D-Grafik, CAD und Spieleentwicklung
- Muss lokal installiert werden → höhere Systemanforderungen
- Integration in Webanwendungen schwierig

Wahl der Rendering API: WebGL

- Läuft direkt im Browser ohne Zusatzsoftware
- Plattformunabhängig durch HTML5-Kompatibilität
- Gut geeignet für interaktive Webanwendungen
- Relativ generisch → hoher Entwicklungsaufwand
- Darstellung meist in separatem Browser-Tab

Wahl der Rendering API: Babylon.js

- 3D-Engine auf Basis von WebGL
- Viele nicht benötigte Funktionen (z.B. Physik, Sound)
- Schnelle Umsetzung komplexer Szenen
- Kann unnötige Komplexität erzeugen
- Darstellung ebenfalls in separatem Browser-Tab

Wahl der Rendering API: Three.js

- Basiert ebenfalls auf WebGL
- Gute Balance zwischen Kontrolle und Abstraktion
- Flexibel, leichtgewichtig und erweiterbar
- Ideal für sich entwickelnde Anforderungen
- Darstellung in separatem Browser-Tab bleibt Einschränkung

Numpy als Mathematik-Library:

- Bessere Performance als SymPy
- symbolische Verarbeitung nicht erforderlich
- Kompatibilität mit SymPy durch Umwandlungen dennoch gegeben

Environment-Klasse

- Wiederkehrende elemente in den Beispiel-Notebooks in einer Klasse zusammengefasst
- Kapselt Erstellung und Verwaltung einer 3D Szene (Kamera, Lichtquelle, Gitter, Achsen) und GUI-Elemente
- einfaches Hinzuzufügen von GUI- und Szenen-Elementen

- Erfolgreiche Integration in JupyterLab: einfache Nutzung und direkte Visualisierung
- Interaktive Bedienung durch ipywidgets (z.B. Schieberegler, Dropdowns)
- Unterstützung verschiedener Rotationsdarstellungen inkl. Umrechnungen
- Simulation radgetriebener Roboter über `move()`-Funktion realisiert
- Kompatibilität mit SymPy sichergestellt
- Zentrale Anforderungen erfüllt. Grundlage für spätere Erweiterungen geschaffen

- Deployment auf den JupyterLab-Servern der FH Dortmund
- Erweiterungen um Stationäre Roboter
- vorwärts- und rückwärts-Transformation